

m00nwalk2

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Revisit the last transmission. We think this transmission contains a hidden message. There are also some clues (clue1, clue2, clue3).

Hints

- Use the clues to extract the another flag from the .wav file
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read audio metadata
 - `sstv` – Decode SSTV images from audio (Python)
 - `steghide` – Extract hidden data from WAV files
 - **Image viewer** – View decoded SSTV images
-

Complete Solution

Step 1: Download and Prepare

Download the audio files from the challenge page:

- `message.wav` – The main transmission containing the hidden flag
- `clue1.wav` , `clue2.wav` , `clue3.wav` – Three clue files to help decode the message

Step 2: Basic File Inspection

First, verify all file types:

```
file clue1.wav clue2.wav clue3.wav message.wav
```

Expected output:

```
clue1.wav:                RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit,
mono 48000 Hz
clue2.wav:                RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit,
mono 48000 Hz
clue3.wav:                RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit,
mono 48000 Hz
message.wav:              RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit,
mono 48000 Hz
```

All files are standard **WAV audio files** – mono, 16-bit, 48 kHz sample rate.

Step 3: Search for Visible Text

Check if the flag appears as plain text within any audio file:

```
strings clue*.wav | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly embedded as text.

Step 4: Inspect Metadata

```
exiftool clue1.wav clue2.wav clue3.wav
```

Output:

```
===== clue1.wav
ExifTool Version Number      : 13.50
File Name                    : clue1.wav
...
Duration                     : 0:02:00
===== clue2.wav
...
Duration                     : 0:01:17
===== clue3.wav
...
Duration                     : 0:01:04
```

Nothing suspicious in the metadata.

Step 5: Recall SSTV from m00nwalk

In the previous challenge ([m00nwalk](#)), we learned that audio files containing beeps and static can be **SSTV (Slow Scan Television)** signals – images encoded as audio.

Let's apply the same technique to the clue files.

Step 6: Install SSTV Decoder

If not already installed:

```
git clone https://github.com/colaclanth/sstv.git
cd sstv
python setup.py install
```

Step 7: Decode the Clue Files

```
sstv -d clue1.wav -o clue1.png
sstv -d clue2.wav -o clue2.png
sstv -d clue3.wav -o clue3.png
```

Output (for each):

```
Reading WAV file...
Decoding SSTV...
Wrote output to: clue1.png
```

Step 8: View the Decoded Images

Open each decoded image:

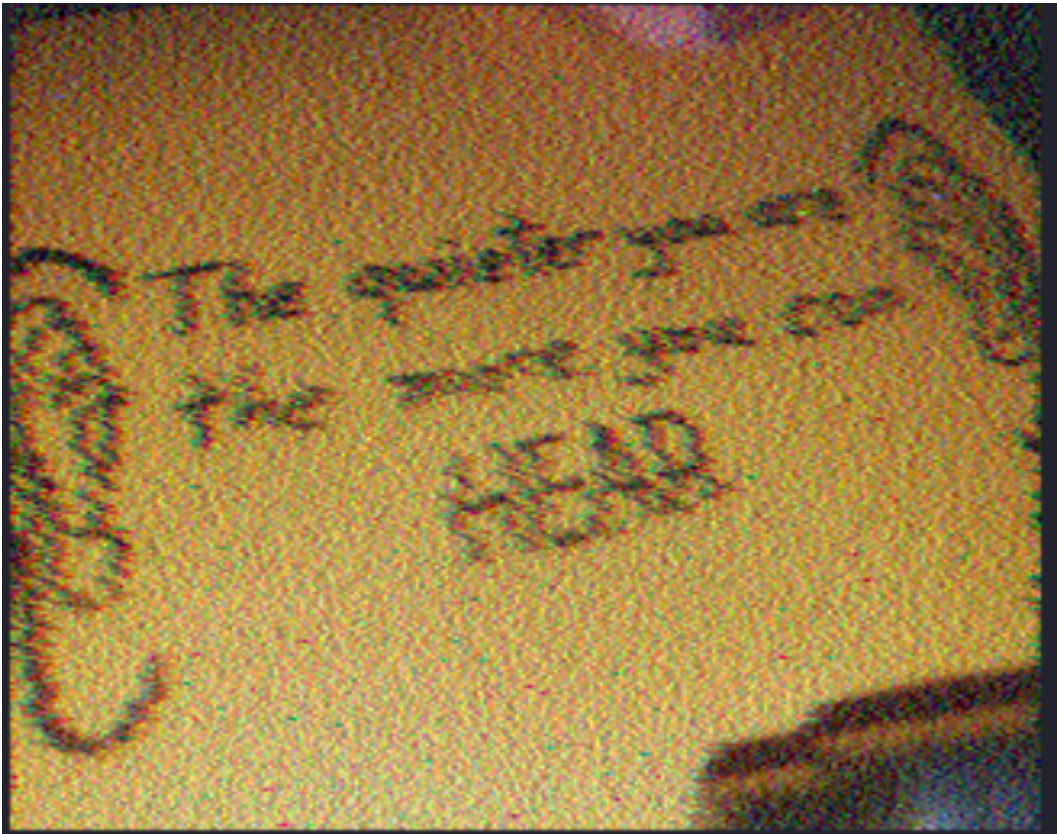
```
xdg-open clue1.png    # Linux
open clue1.png        # macOS
start clue1.png       # Windows
```

Results:

File	Image Content
clue1.png	A partial flag or hint – likely part of a password



File	Image Content
clue2.png	The second piece



File	Image Content
clue3.png	The third piece



Step 9: Combine the Clues

When you combine the information from all three decoded images, they reveal a **password** (e.g., `<REDACTED>`).

Note: The three clue images together form a complete passphrase. You may need to piece them together manually or overlay them.

Step 10: Extract Hidden Data from message.wav

Now that we have the password, we can use `steghide` to extract hidden data from the main `message.wav` file.

`steghide` is a steganography tool that can hide data inside WAV files (as well as JPEG, BMP, AU).

```
steghide extract -sf message.wav -p <REDACTED>
```

Output:

```
wrote extracted data to "steganoplayload12154.txt".
```

Step 11: Read the Extracted Flag

```
cat steganoplayload12154.txt
```

Output:

```
picoCTF{<REDACTED>}
```

Step 12: Alternative Method – Using QSSTV (GUI)

If you prefer a graphical interface for SSTV decoding:

```
sudo apt install qsstv
```

Then:

- 1. Launch QSSTV
- 2. Go to **File** → **Playback** → **Play**
- 3. Select each `clue*.wav` file
- 4. The decoded images will appear in the main window
- 5. Combine the clues to get the password

Step 13: Alternative Method – Using Sonic Visualiser

You can also use **Sonic Visualiser** to examine the spectrograms:

```
sudo apt install sonic-visualiser
```

Open each clue file and view the spectrogram to see the hidden images.

 Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, decoding the SSTV clues and using the resulting password with `steghide` will reveal the complete flag.

 Key Concepts

Concept	Description
SSTV (Slow Scan Television)	A method of transmitting images via audio frequencies.
Steganography	Hiding data within other files (e.g., hiding a text file inside a WAV audio file).

Concept	Description
steghide	A command-line tool for embedding and extracting hidden data in WAV, JPEG, BMP, and AU files.
Multi-stage Decoding	Some challenges require multiple steps: decode SSTV → get password → use steghide → get flag.
Clue Files	Supplementary files that provide hints or partial information needed to solve the main challenge.

Pro Tips

- Always decode clue files first** – The challenge provides multiple `.wav` files. Process all of them.
- Combine visual information** – The three SSTV images may form a single complete password when overlaid or read together.
- steghide works with WAV files** – Many people forget this. `steghide` supports WAV, AU, JPEG, and BMP.
- Check file sizes** – `message.wav` is the largest file (11+ MB) – perfect for hiding a payload.
- The password may be case-sensitive** – Enter it exactly as shown in the combined clues.
- Try different SSTV modes** – If one decoder doesn't work well, try QSSTV or RxSSTV.

Alternative Steghide Commands

Command	Purpose
<code>steghide info message.wav</code>	Check if the file contains embedded data
<code>steghide extract -sf message.wav</code>	Extract without password (if no password is set)

Command	Purpose
<code>steghide embed -cf message.wav -ef flag.txt -p <password></code>	Embed a file into a WAV

Alternative SSTV Decoders

Tool	Platform	Command/Link
sstv (Python)	Linux/macOS/Windows	<code>pip install sstv</code> or GitHub repo
QSSTV	Linux	<code>sudo apt install qsstv</code>
RxSSTV	Windows	Download from onthesystem.com
MMSSTV	Windows	Popular SSTV decoder for amateur radio
Sonic Visualiser	Linux/macOS/Windows	Spectrogram analysis